Practica 3 — Categorias y Relaciones

Agregar el modelo Category y relacionarlo con Product mediante una relacion Foreign Key, sobre el proyecto tienda_online de la Practica 2.

Autor: aaron03042005@gmail.com
Fecha: 23/09/2026
Proyecto: tienda_online

Contenido

1. Que se hizo
2. 1. models.py
3. 2. admin.py
4. 3. serializers.py
5. 4. views.py
6. 5. urls.py
7. 6. Migraciones
8. 7. Nombre de categoria en la API
9. 8. Filtro por categoria
10. Capturas del admin y de la API

Que se hizo

El modelo **Category**, la relacion **ForeignKey** hacia **Product**, su registro en el administrador, su serializador, su vista y su ruta en el router ya venian del trabajo hecho en la Practica 2. Para esta practica se agregaron dos cosas nuevas que no estaban antes: que la API muestre el nombre de la categoria

en vez de solo su id, y que se pueda filtrar la lista de productos por categoria desde la URL. Estos dos cambios se explican en los puntos 7 y 8.

1. models.py

Se agrego la clase **Category**, con un campo **name**. En el modelo **Product** se agrego el campo **category** como **ForeignKey** hacia **Category**, con **on_delete=models.CASCADE**, de forma que cada producto queda ligado a una categoria y, si se borra una categoria, se borran tambien sus productos.

2. admin.py

Se registro el modelo **Category** con **admin.site.register(Category)** para que apareciera en el panel de administracion junto a **Product**, que ya estaba registrado.

3. serializers.py

Se agrego **CategorySerializer**, un **ModelSerializer** sobre el modelo **Category** con todos sus campos, para que la API pudiera devolver las categorias en formato JSON.

4. views.py

Se agrego **CategoryViewSet**, un **ModelViewSet** que usa **CategorySerializer** y expone el CRUD completo (crear, leer, actualizar, borrar) sobre las categorias.

5. urls.py

Se registro la ruta **categories** en el **DefaultRouter** del proyecto, junto a la ruta **products** que ya existia, quedando ambas expuestas bajo **/api/**.

6. Migraciones

Se corrio **python manage.py makemigrations** y **python manage.py migrate** para aplicar el modelo **Category** y la relacion **ForeignKey** en **Product** sobre la base de datos MySQL. Como el modelo ya estaba migrado desde la Practica 2, Django confirmo que no habia cambios pendientes, sin necesidad de borrar la base de datos ni pedir un valor por default.

7. Nombre de categoria en la API

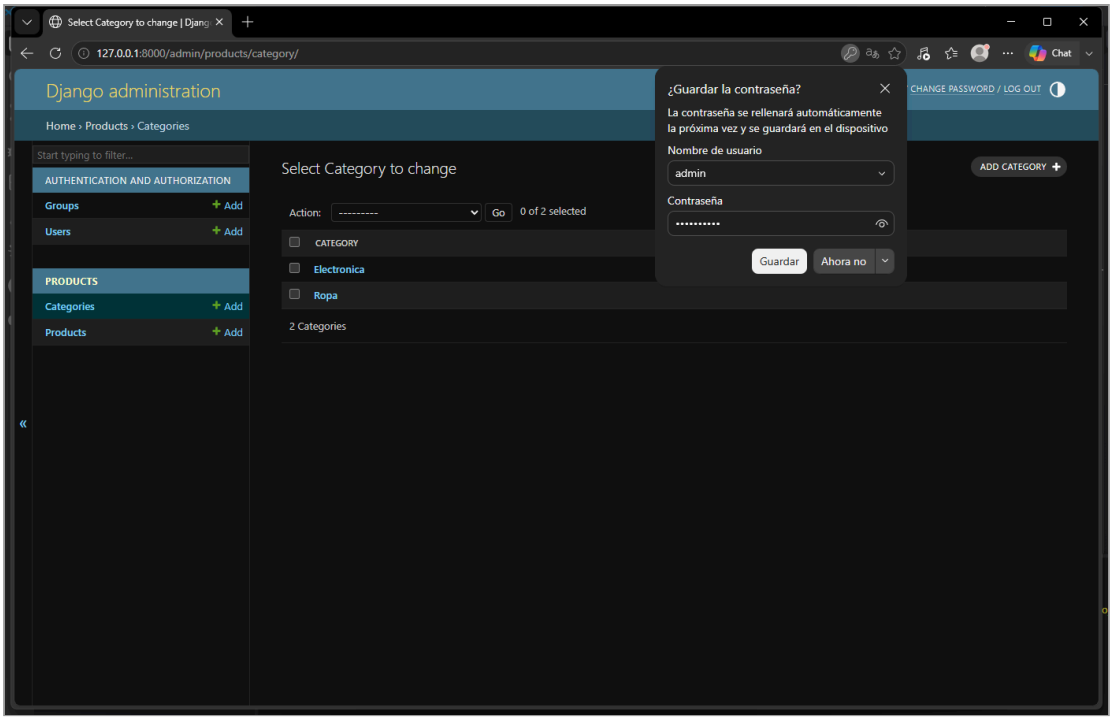
Antes, el campo **category** en **/api/products/** mostraba solo el id de la categoria (por ejemplo **1**). Se agrego **category = serializers.StringRelatedField()** al inicio de **ProductSerializer**, en **serializers.py**, para que ese campo muestre el nombre de la categoria (**Electronica**, **Ropa**) en vez de su id.

8. Filtro por categoria

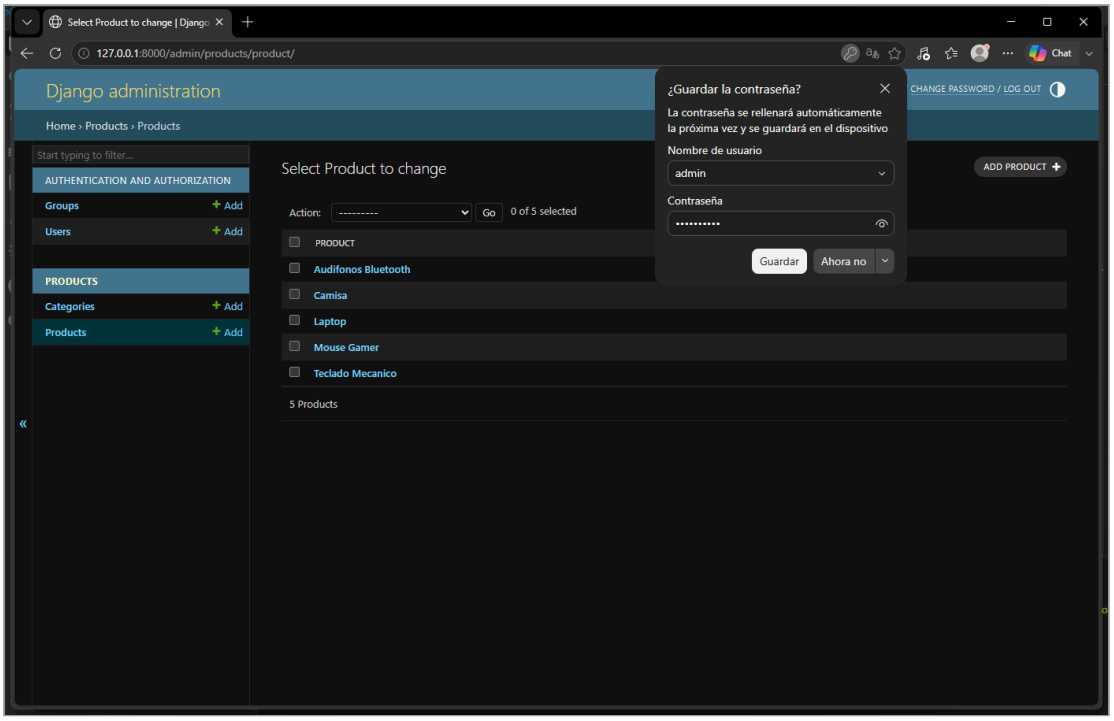
Se agrego **from django_filters.rest_framework import DjangoFilterBackend** en **views.py**, y en **ProductViewSet** se agregaron las lineas **filter_backends = [DjangoFilterBackend]** y **filterset_fields = ['category']**. Con esto, entrar a **/api/products/?category=1** devuelve unicamente los productos de esa categoria.

Capturas del admin y de la API

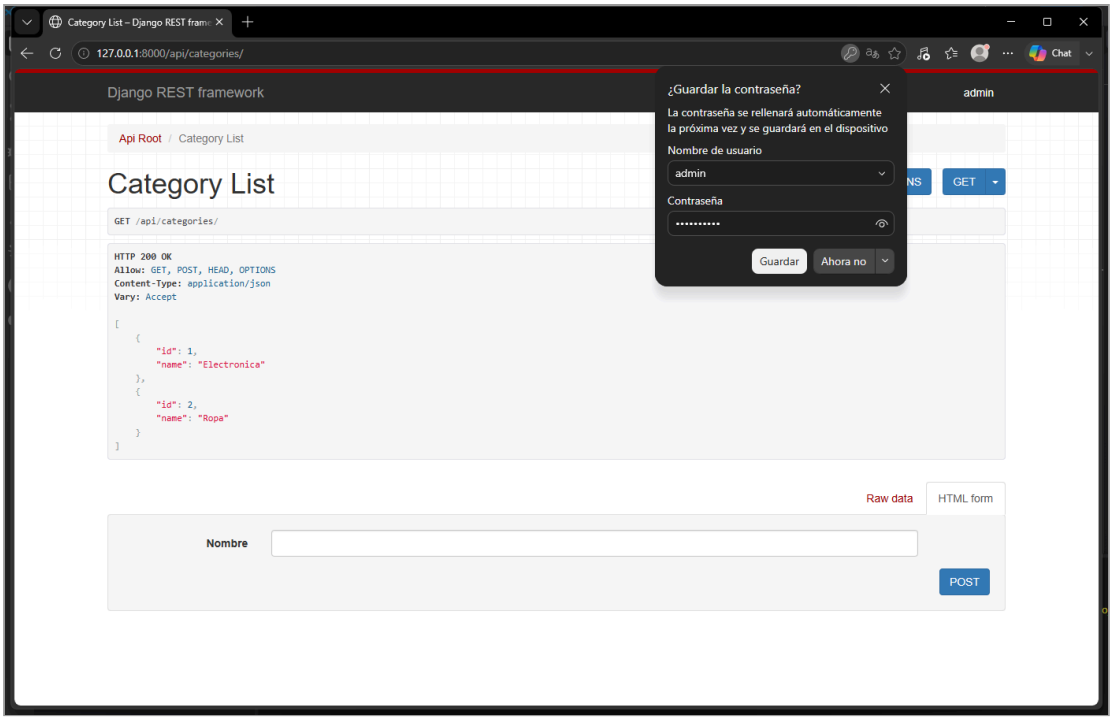
Panel de administracion mostrando las categorias:



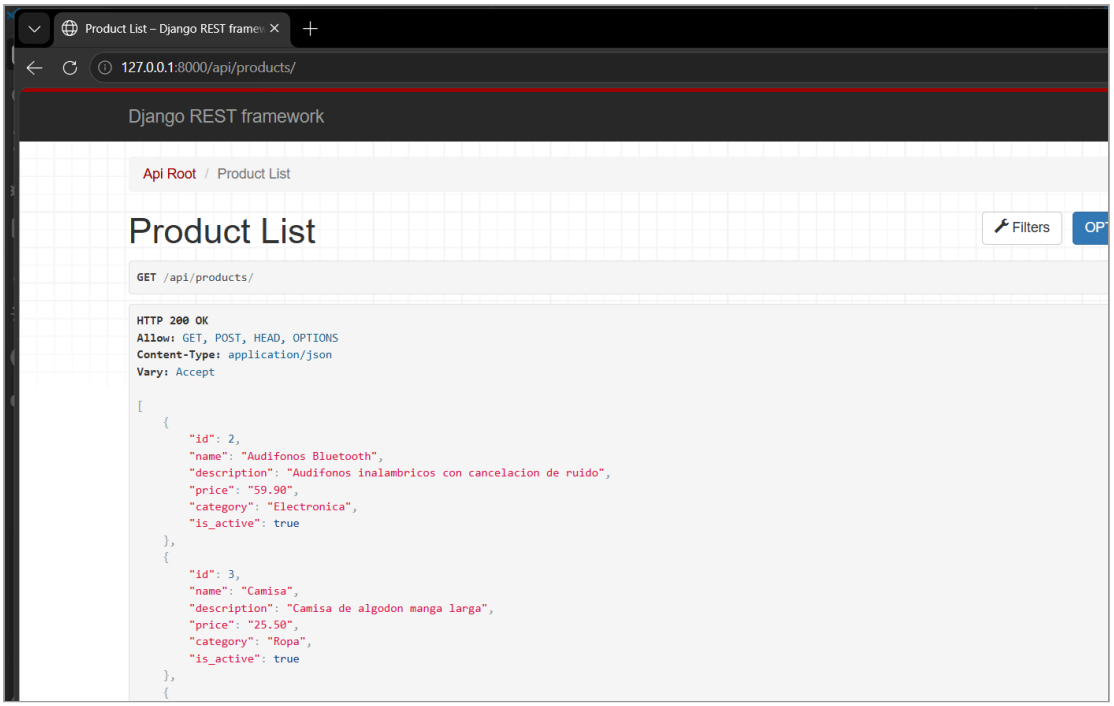
Panel de administracion mostrando los productos, cada uno ligado a una categoria:



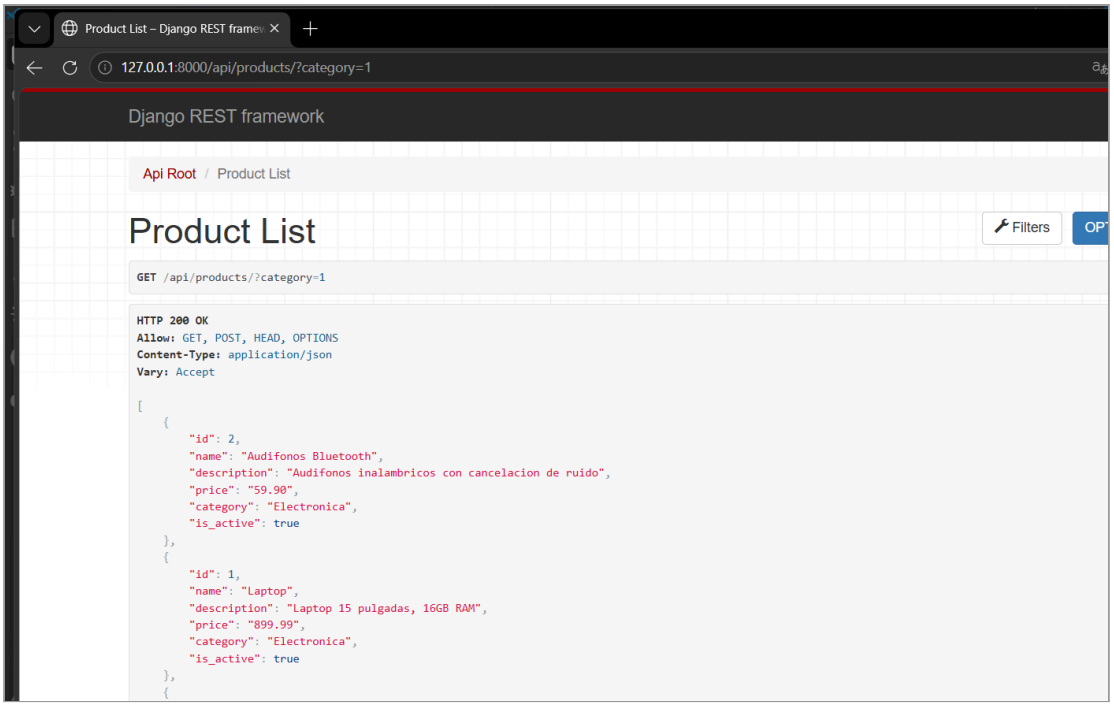
API en **/api/categories/**:



API en **/api/products/**, con el nombre de la categoria visible en cada producto:



API en **/api/products/?category=1**, con el filtro por categoria funcionando:


Product List – Django REST frame
127.0.0.1:8000/api/products/?category=1
Django REST framework
Api Root / Product List
Product List
Filters
GET /api/products/?category=1
HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept
[
{
"id": 2,
"name": "Audifonos Bluetooth",
"description": "Audifonos inalambricos con cancelacion de ruido",
"price": "59.90",
"category": "Electronica",
"is_active": true
},
{
"id": 1,
"name": "Laptop",
"description": "Laptop 15 pulgadas, 16GB RAM",
"price": "899.99",
"category": "Electronica",
"is_active": true
},
{